



# Guía de Deployment - Horizon Medical

**Versión:** 2.0  
**Última actualización:** Enero 2026  
**Audiencia:** DevOps, SRE, Ingenieros de Infraestructura

## Tabla de Contenidos

- 1. [Requisitos del Servidor](#)
- 2. [Instalación y Configuración](#)
- 3. [Variables de Entorno](#)
- 4. [Configuración de Bases de Datos](#)
- 5. [Deployment en Producción](#)
- 6. [Docker y Containerización](#)
- 7. [Kubernetes](#)
- 8. [CI/CD Pipeline](#)
- 9. [Monitoreo y Logging](#)
- 10. [Backup y Recuperación](#)
- 11. [Scaling y Performance](#)
- 12. [Security y Compliance](#)

## Requisitos del Servidor

### Requisitos de Hardware (Mínimo)

**Para Desarrollo:**

|                 |            |
|-----------------|------------|
| CPU:            | 4 cores    |
| RAM:            | 8 GB       |
| Almacenamiento: | 100 GB SSD |
| Ancho de banda: | 10 Mbps    |
| Uptime:         | 99%        |

**Para Producción (Pequeña):**

|                 |          |
|-----------------|----------|
| CPU:            | 16 cores |
| RAM:            | 64 GB    |
| Almacenamiento: | 1 TB SSD |
| Ancho de banda: | 100 Mbps |
| Uptime:         | 99.9%    |
| Redundancia:    | 2 nodos  |

**Para Producción (Mediana):**

CPU: 32 cores  
RAM: 256 GB  
Almacenamiento: 5 TB SSD  
Ancho de banda: 1 Gbps  
Uptime: 99.99%  
Redundancia: 3+ nodos  
Regiones: Multi-región

## Requisitos de Software

Sistema Operativo: Ubuntu 20.04 LTS / 22.04 LTS  
o Amazon Linux 2  
o CentOS 8+

Runtime: Node.js 18+ LTS  
Python 3.10+  
Docker 20.10+  
Docker Compose 2.x  
Kubernetes 1.24+

Herramientas: kubectl 1.24+  
helm 3.x  
terraform 1.x  
git 2.x

---

## Instalación y Configuración

### Preparación del Servidor

#### 1. Actualizar el Sistema

```
sudo apt update && sudo apt upgrade -y

# Instalar dependencias básicas
sudo apt install -y \
  curl wget git build-essential \
  openssl ca-certificates \
  apt-transport-https software-properties-common
```

#### 2. Instalar Node.js

```
# Agregar repositorio Node.js
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

# Instalar Node.js
sudo apt install -y nodejs

# Verificar instalación
node --version    # v20.x.x
npm --version     # 10.x.x
```

### 3. Instalar Docker

```
# Agregar clave GPG
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# Agregar repositorio
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu \
    $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Instalar Docker
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Agregar usuario al grupo docker
sudo usermod -aG docker $USER
newgrp docker

# Verificar
docker --version
```

### 4. Instalar Kubernetes (kubectl)

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

kubectl version --client
```

## Instalación de Dependencias del Proyecto

```
# Clonar repositorio
git clone https://github.com/horizon-medical/horizon-medical.git
cd horizon-medical

# Instalar dependencias de Backend
cd backend
npm install

# Instalar dependencias de Frontend
cd ../web
npm install

# Instalar dependencias de Mobile
cd ../mobile
npm install

# Volver al root
cd ..
```

## Variables de Entorno

---

### Archivo .env (Backend)

```

# Aplicación
NODE_ENV=production
PORT=3000
LOG_LEVEL=info
DEBUG=false

# Base de Datos
MONGODB_URI=mongodb+srv://user:password@cluster.mongodb.net/horizon_prod
MONGODB_POOL_SIZE=20

POSTGRES_HOST=postgres-prod.horizon-medical.com
POSTGRES_PORT=5432
POSTGRES_DB=horizon_medical
POSTGRES_USER=horizon_app
POSTGRES_PASSWORD=securepassword123
POSTGRES_POOL_MIN=5
POSTGRES_POOL_MAX=20

REDIS_URL=redis://:password@redis-prod.horizon-medical.com:6379/0
REDIS_POOL_SIZE=10

TIMESCALEDB_URL=postgresql://user:password@timescaledb-prod.horizon-medical.com:5432/
horizon_timeseries

ELASTICSEARCH_HOST=elasticsearch-prod.horizon-medical.com
ELASTICSEARCH_PORT=9200
ELASTICSEARCH_USER=elastic
ELASTICSEARCH_PASSWORD=securepassword

# Autenticación
JWT_SECRET=your_jwt_secret_key_min_32_chars
JWT_EXPIRATION=3600
REFRESH_TOKEN_SECRET=your_refresh_token_secret_min_32_chars
REFRESH_TOKEN_EXPIRATION=604800

# OAuth
OAUTH_GOOGLE_CLIENT_ID=your_client_id.apps.googleusercontent.com
OAUTH_GOOGLE_CLIENT_SECRET=your_client_secret

# Seguridad
ENCRYPTION_KEY=your_32_char_encryption_key
ENCRYPTION_ALGORITHM=aes-256-cbc

# AWS
AWS_REGION=eu-west-1
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
AWS_S3_BUCKET=horizon-medical-prod
AWS_S3_URL_EXPIRATION=3600

# Servicios Externos
SENDGRID_API_KEY=SG.aaaaaaaaaaaaa
SENDGRID_FROM_EMAIL=noreply@horizon-medical.com

TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your_twilio_token
TWILIO_PHONE_NUMBER=+34912345678

STRIPE_API_KEY=sk_live_xxxxxxxxxxxxxx
STRIPE_WEBHOOK_SECRET=whsec_xxxxxxxxxxxxxx

# HIPAA/Compliance

```

```

HIPAA_ENABLED=true
AUDIT_LOG_ENABLED=true
DATA_RETENTION_DAYS=2555 # 7 years
BACKUP_FREQUENCY=daily

# Performance
MAX_CONCURRENT_SESSIONS=10000
SESSION_SYNC_INTERVAL=30000 # 30 seconds
RATE_LIMIT_REQUESTS=10000
RATE_LIMIT_WINDOW=3600

# Logging
SENTRY_DSN=https://xxxxx@xxxxx.ingest.sentry.io/xxxxx
LOG_DRIVER=winston
LOG_FORMAT=json
LOG_RETENTION_DAYS=90

# Email
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASS=SG.xxxxxxxxxxxxxx

# Alertas
ALERT_WEBHOOK_URL=https://your-webhook-endpoint.com/alerts
CRITICAL_ALERT_SMS=true
CRITICAL_ALERT_EMAIL=true
CRITICAL_ALERT_PHONE=false

```

## Archivo .env (Frontend)

```

# API
VITE_API_BASE_URL=https://api.horizon-medical.com/v2
VITE_API_TIMEOUT=30000
VITE_WEBSOCKET_URL=wss://api.horizon-medical.com

# Autenticación
VITE_AUTH_REDIRECT_URI=https://dashboard.horizon-medical.com/auth/callback
VITE_OAUTH_GOOGLE_CLIENT_ID=your_client_id.apps.googleusercontent.com

# Analítica
VITE_ANALYTICS_ID=G-XXXXXXXXXX
VITE_SENTRY_DSN=https://xxxxx@xxxxx.ingest.sentry.io/xxxxx

# Features
VITE_ENABLE_OFFLINE_MODE=true
VITE_ENABLE_DARK_MODE=true
VITE_ENABLE_MONITORING=true

# URLs
VITE_APP_URL=https://dashboard.horizon-medical.com
VITE_SUPPORT_URL=https://help.horizon-medical.com
VITE_PRIVACY_URL=https://horizon-medical.com/privacy
VITE_TERMS_URL=https://horizon-medical.com/terms

```

## Archivo .env (Mobile)

```
# API
REACT_APP_API_BASE_URL=https://api.horizon-medical.com/v2
REACT_APP_WEBSOCKET_URL=wss://api.horizon-medical.com

# Bluetooth
BLUETOOTH_SERVICE_UUID=180D
BLUETOOTH_CHAR_UUID=2A37

# Almacenamiento
STORAGE_PATH=/data/horizon_medical
MAX_OFFLINE_DATA_GB=5

# Logging
LOG_LEVEL=info
CRASH_REPORTING=true
```

---

## Configuración de Bases de Datos

### MongoDB Setup

```
# Crear base de datos
db.createDatabase("horizon_medical")

# Crear usuario
db.createUser({
  user: "horizon_app",
  pwd: "securepassword123",
  roles: [
    { role: "readWrite", db: "horizon_medical" },
    { role: "dbAdmin", db: "horizon_medical" }
  ]
})

# Crear índices
use horizon_medical

db.patients.createIndex({ email: 1 }, { unique: true })
db.patients.createIndex({ createdAt: -1 })
db.sessions.createIndex({ patientId: 1, createdAt: -1 })
db.sessions.createIndex({ startTime: 1, endTime: 1 })
db.devices.createIndex({ serialNumber: 1 }, { unique: true })
db.devices.createIndex({ patientId: 1 })
db.alerts.createIndex({ patientId: 1, severity: 1, createdAt: -1 })
```

## PostgreSQL Setup

```
-- Crear base de datos
CREATE DATABASE horizon_medical;

-- Conectar a la base de datos
\c horizon_medical

-- Crear usuario
CREATE USER horizon_app WITH PASSWORD 'securepassword123';

-- Crear extensiones
CREATE EXTENSION uuid-oss;
CREATE EXTENSION pg_trgm; -- Para búsqueda full-text
CREATE EXTENSION hstore;

-- Conceder permisos
GRANT ALL PRIVILEGES ON DATABASE horizon_medical TO horizon_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON TABLES TO horizon_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON SEQUENCES TO horizon_app;
```

## Redis Setup

```
# Configurar Redis
cat > /etc/redis/redis.conf << EOF
port 6379
bind 127.0.0.1 ::1
requirepass securepassword123
maxmemory 8gb
maxmemory-policy allkeys-lru
appendonly yes
appendfilename "appendonly.aof"
EOF

# Iniciar Redis
sudo systemctl start redis-server
sudo systemctl enable redis-server

# Verificar
redis-cli ping # Debería responder: PONG
```

## TimescaleDB Setup

```
-- Conectar a PostgreSQL primero
psql -U postgres

-- Crear extensión
CREATE EXTENSION IF NOT EXISTS timescaledb CASCADE;

-- Crear tabla hipertemporal
CREATE TABLE ecg_data (
  time TIMESTAMPTZ NOT NULL,
  session_id UUID NOT NULL,
  device_id UUID NOT NULL,
  derivation VARCHAR(5) NOT NULL,
  value FLOAT NOT NULL,
  heart_rate INT
);

-- Crear hypertable
SELECT create_hypertable('ecg_data', 'time', if_not_exists => TRUE);

-- Crear índices
CREATE INDEX idx_ecg_session_time ON ecg_data (session_id, time DESC);
CREATE INDEX idx_ecg_device_time ON ecg_data (device_id, time DESC);
```

---

## Deployment en Producción

### Pasos de Deployment

#### 1. Preparación

```
# Verificar código
npm run lint
npm run test
npm run build

# Verificar seguridad
npm audit

# Crear tag de versión
git tag -a v2.0.0 -m "Production release v2.0.0"
git push origin v2.0.0
```

## 2. Build

```
# Backend build
cd backend
npm run build
cd ..

# Frontend build
cd web
npm run build
cd ..

# Mobile build
cd mobile
ionic build --prod
cd ..
```

## 3. Deployment Manual (sin K8s)

```
# En servidor de producción
cd /var/www/horizon-medical

# Descargar nueva versión
git fetch --all
git checkout v2.0.0

# Actualizar dependencias
npm ci --production

# Ejecutar migraciones
npm run migrate

# Reiniciar aplicación
sudo systemctl restart horizon-medical

# Verificar estado
sudo systemctl status horizon-medical
```

## 4. Rollback

```
# Volver a versión anterior
git checkout v1.9.9

# Reinstalar dependencias
npm ci

# Ejecutar migraciones hacia atrás
npm run migrate:down

# Reiniciar
sudo systemctl restart horizon-medical
```

---

# Docker y Containerización

---

## Dockerfile (Backend)

```
FROM node:20-alpine

WORKDIR /app

# Copiar package files
COPY package*.json ./

# Instalar dependencias
RUN npm ci --production

# Copiar código
COPY src ./src

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
  CMD node -e "require('http').get('http://localhost:3000/health', (r) => {if (r.statusCode !== 200) throw new Error(r.statusCode)})"

# Exponer puerto
EXPOSE 3000

# Iniciar aplicación
CMD ["npm", "start"]
```

## **Docker Compose (Completo)**

```

version: '3.9'

services:
  backend:
    build: ./backend
    container_name: horizon-backend
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - MONGODB_URI=mongodb://mongo:27017/horizon_medical
      - POSTGRES_HOST=postgres
      - REDIS_URL=redis://redis:6379
    depends_on:
      - mongo
      - postgres
      - redis
    volumes:
      - ./backend/logs:/app/logs
    restart: unless-stopped

  web:
    build:
      context: ./web
      target: production
    container_name: horizon-web
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./web/nginx.conf:/etc/nginx/nginx.conf
      - ./certs:/etc/nginx/certs
    depends_on:
      - backend
    restart: unless-stopped

  mongo:
    image: mongo:6.0
    container_name: horizon-mongo
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: password
      MONGO_INITDB_DATABASE: horizon_medical
    volumes:
      - mongo-data:/data/db
      - ./init-mongo.sh:/docker-entrypoint-initdb.d/init-mongo.sh
    restart: unless-stopped

  postgres:
    image: postgres:15-alpine
    container_name: horizon-postgres
    ports:
      - "5432:5432"
    environment:
      POSTGRES_PASSWORD: password
      POSTGRES_DB: horizon_medical
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./init-postgres.sql:/docker-entrypoint-initdb.d/init.sql
    restart: unless-stopped

```

```

redis:
  image: redis:7-alpine
  container_name: horizon-redis
  ports:
    - "6379:6379"
  command: redis-server --appendonly yes
  volumes:
    - redis-data:/data
  restart: unless-stopped

elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.5.0
  container_name: horizon-elasticsearch
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
  ports:
    - "9200:9200"
  volumes:
    - elasticsearch-data:/usr/share/elasticsearch/data
  restart: unless-stopped

volumes:
  mongo-data:
  postgres-data:
  redis-data:
  elasticsearch-data:

networks:
  default:
    name: horizon-network

```

## Construir y Ejecutar

```

# Construir imágenes
docker-compose build

# Ejecutar servicios
docker-compose up -d

# Verificar estado
docker-compose ps

# Ver logs
docker-compose logs -f backend

# Detener
docker-compose down

```

---

# Kubernetes

---

## Namespace Setup

```
apiVersion: v1
kind: Namespace
metadata:
  name: horizon-medical
```

## Backend Deployment

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: horizon-backend
  namespace: horizon-medical
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: horizon-backend
  template:
    metadata:
      labels:
        app: horizon-backend
    spec:
      containers:
      - name: backend
        image: horizon-medical/backend:v2.0.0
        ports:
        - containerPort: 3000
        env:
        - name: NODE_ENV
          value: "production"
        - name: MONGODB_URI
          valueFrom:
            secretKeyRef:
              name: horizon-secrets
              key: mongodb-uri
      resources:
        requests:
        cpu: 500m
        memory: 512Mi
        limits:
        cpu: 2000m
        memory: 2Gi
      livenessProbe:
        httpGet:
        path: /health
        port: 3000
        initialDelaySeconds: 30
        periodSeconds: 10
      readinessProbe:
        httpGet:
        path: /ready
        port: 3000
        initialDelaySeconds: 5
        periodSeconds: 5
      affinity:
        podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:

```

```
- horizon-backend
topologyKey: kubernetes.io/hostname
```

## Service

```
apiVersion: v1
kind: Service
metadata:
  name: horizon-backend-svc
  namespace: horizon-medical
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 3000
      protocol: TCP
  selector:
    app: horizon-backend
```

## Apply en Cluster

```
# Crear namespace
kubectl apply -f namespace.yaml

# Crear secrets
kubectl create secret generic horizon-secrets \
  --from-literal=mongodb-uri='mongodb://...' \
  -n horizon-medical

# Aplicar configuraciones
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml

# Verificar estado
kubectl get pods -n horizon-medical
kubectl get svc -n horizon-medical
kubectl logs -n horizon-medical -l app=horizon-backend
```

---

## CI/CD Pipeline

### GitHub Actions ( `.github/workflows/deploy.yml` )

```

name: Deploy to Production

on:
  push:
    tags:
      - 'v*'

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Run Tests
        run: |
          npm ci
          npm run test:coverage

      - name: Build Docker Images
        run: |
          docker build -t horizon-backend:${{ github.ref_name }} ./backend
          docker build -t horizon-web:${{ github.ref_name }} ./web

      - name: Push to Registry
        env:
          REGISTRY: ghcr.io
          REGISTRY_USERNAME: ${GITHUB_ACTOR}
          REGISTRY_PASSWORD: ${GITHUB_TOKEN}
        run: |
          echo $REGISTRY_PASSWORD | docker login $REGISTRY -u $REGISTRY_USERNAME --
password-stdin
          docker push $REGISTRY/horizon-medical/backend:${{ github.ref_name }}
          docker push $REGISTRY/horizon-medical/web:${{ github.ref_name }}

      - name: Deploy to K8s
        run: |
          kubectl set image deployment/horizon-backend \
            horizon-backend=ghcr.io/horizon-medical/backend:${{ github.ref_name }} \
            -n horizon-medical
          kubectl rollout status deployment/horizon-backend -n horizon-medical

```

# Monitoreo y Logging

---

## Prometheus Configuration

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'horizon-backend'
    static_configs:
      - targets: ['localhost:3000']

  - job_name: 'horizon-mongodb'
    static_configs:
      - targets: ['localhost:27017']

  - job_name: 'horizon-postgres'
    static_configs:
      - targets: ['localhost:5432']
```

## ELK Stack (Elasticsearch, Logstash, Kibana)

```
# Logstash configuration
input {
  tcp {
    port => 5000
    codec => json
  }
}

filter {
  if [type] == "app-log" {
    grok {
      match => { "message" => "%{TIMESTAMP_ISO8601:timestamp} %{LOGLEVEL:level} %
{DATA:message}" }
    }
  }
}

output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "horizon-logs-%{+YYYY.MM.dd}"
  }
}
```

# Backup y Recuperación

---

## Script de Backup

```
#!/bin/bash

BACKUP_DIR="/backup/horizon-medical"
DATE=$(date +%Y%m%d_%H%M%S)
MONGODB_BACKUP="$BACKUP_DIR/mongodb_$DATE.tar.gz"
POSTGRES_BACKUP="$BACKUP_DIR/postgres_$DATE.tar.gz"

# Backup MongoDB
mongodump --out /tmp/mongo_backup_$DATE
tar -czf $MONGODB_BACKUP -C /tmp mongo_backup_$DATE
rm -rf /tmp/mongo_backup_$DATE

# Backup PostgreSQL
pg_dump horizon_medical | gzip > $POSTGRES_BACKUP

# Backup S3
aws s3 sync $BACKUP_DIR s3://horizon-backups/

# Limpieza (mantener últimos 30 días)
find $BACKUP_DIR -mtime +30 -delete

# Verificar integridad
tar -tzf $MONGODB_BACKUP > /dev/null && echo "MongoDB backup OK" || echo "MongoDB backup FAILED"
gunzip -t $POSTGRES_BACKUP && echo "PostgreSQL backup OK" || echo "PostgreSQL backup FAILED"
```

## Restore Procedure

```
# Restaurar MongoDB
tar -xzf mongodb_$DATE.tar.gz
mongorestore mongo_backup_$DATE

# Restaurar PostgreSQL
psql horizon_medical < backup.sql
```

---

## Scaling y Performance

### Auto-Scaling Configuration (K8s)

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: horizon-backend-hpa
  namespace: horizon-medical
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: horizon-backend
  minReplicas: 3
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80

```

## Security y Compliance

### SSL/TLS Configuration

```

# Generar certificado Let's Encrypt
certbot certonly --standalone -d api.horizon-medical.com -d dashboard.horizon-medical.com

# Configuración Nginx
server {
    listen 443 ssl http2;
    server_name api.horizon-medical.com;

    ssl_certificate /etc/letsencrypt/live/api.horizon-medical.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.horizon-medical.com/privkey.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;
}

```

## Secrets Management

```
# Con Kubernetes Secrets
kubectl create secret generic db-credentials \
  --from-literal=username=admin \
  --from-literal=password=securepass \
  -n horizon-medical

# Con HashiCorp Vault (recomendado)
vault kv put secret/horizon/database \
  username=admin \
  password=securepass
```

---

**Versión:** 2.0

**Última actualización:** Enero 2026

**Próxima revisión:** Julio 2026

Para soporte de infraestructura: [infrastructure@horizon-medical.com](mailto:infrastructure@horizon-medical.com)